Food Recognition and Ingredient Detection via Lightweight Neural Networks with BLIP Pseudo-Labelling

Luyichun
yichunlu707@gmail.com

Abstract

Automatic food recognition has broad applications in dietary tracking, allergy management, and nutritional awareness [15]. While large vision-language models (VLMs) such as BLIP and GPT-4V can describe food images in rich natural language, their vast memory footprint and high inference latency make them impractical for deployment on resource-constrained consumer devices or as real-time services.

To bridge this gap, multi-label learning has frequently been deployed to untangle the overlapping visual features of multi-ingredient dishes [2]. This work proposes a lightweight two-model inference pipeline that leverages BLIP solely as a one-time offline pseudo-label generator, distilling its foundational knowledge into a compact IngredientNet classifier that operates entirely without a VLM at inference time.

Experiments on the Food-101 dataset demonstrate that our per-image visual labeling strategy significantly outperforms traditional class-level baselines (F1: 0.2947 vs. 0.0115), and a thorough threshold sensitivity analysis identifies 0.2 as the optimal decision boundary. Crucially, the deployed system requires approximately 50 MB of memory — achieving a 40× reduction compared to keeping a VLM in the active inference pipeline, demonstrating a practical approach to localized, on-device dietary AI.

1. Introduction

This project proposes a lightweight, two-model inference pipeline designed to achieve both accurate food-type classification and multi-label ingredient detection. Crucially, the architecture eliminates the need for large language-vision models (LVMs) or vision-language models (VLMs) during inference, avoiding the severe computational bottlenecks typically associated with deploying such models on consumer hardware [15].

The core methodology leverages the Bootstrapping Language-Image Pre-training (BLIP) model [7] strictly as a **one-time pseudo-label generator** during the dataset construction phase [6]. This rich knowledge is subsequently distilled into a compact convolutional neural network (*IngredientNet*) optimized for millisecond-range execution at the edge.

1.1. Inference Pipeline Components

The final end-to-end system processes a single food image input through parallel tracks to generate three distinct outputs:

- **Food-Type Classification:** A fine-tuned *EfficientNet-B0* classifier [16] maps the input image to one of 101 discrete culinary categories [1], utilizing robust deep convolutional feature representations [18].
- **Visual Ingredient Detection:** A custom multi-label convolutional neural network (*IngredientNet*), trained on the BLIP-generated pseudo-labels, predicts the presence of multiple concurrent ingredients [2].
- **Audio Report Synthesis:** The classification and ingredient outputs are aggregated into a coherent natural language report, which is then synthesized into speech using the lightweight *edge-tts* engine, enabling real-time multimodal feedback without relying on heavy cloud-based text-to-speech APIs.

1.2. The Key Contributions Of This Work

- **Resource-efficient pseudo-labelling:** *BLIP* is used only once offline to annotate training images. At inference, the deployed model is a 5.3M-parameter CNN with no dependency on any VLM, reducing GPU memory requirements from ∼4 GB (BLIP) to under 100 MB, demonstrating a viable path toward fully localized, on-device dietary AI.
- **Per-image visual labels:** Unlike knowledge-base approaches that assign a static, predetermined ingredient list to every image of the same food category, *BLIP* annotates each image individually based on actual visual content. This enables the downstream classifier to learn image-specific features and handle real-world variations in multi-ingredient dishes.
- **Pseudo-label cleaning pipeline:** A dedicated post-processing stage removes hallucinated, repetitive, and

low-frequency tokens from *BLIP* outputs before training, effectively combating the standard data-noise issues inherent to automated zero-shot labeling pipelines [11] without requiring manual human annotation.

2. Related Works

2.1. Food Image Classification

Food recognition has been studied extensively since the introduction of the Food-101 benchmark [1], which contains 101,000 images across 101 categories. Early approaches relied on hand-crafted features; modern methods achieve over 90% top-1 accuracy using deep residual networks and vision transformers. EfficientNet [16] offers a strong accuracy-efficiency trade-off and is a common backbone for food classification tasks.

2.2. Transfer Learning and Fine-Tuning

Transfer learning from ImageNet pre-trained weights has become standard practice in food recognition [18]. Differential fine-tuning—unfreezing only the top layers of a pre-trained backbone while keeping lower layers frozen—allows high-level task-specific features to be learned while preserving low-level texture representations that transfer well from ImageNet.

2.3. Vision-Language Models and BLIP

BLIP (Bootstrapping Language-Image Pre-training) [7] is a multimodal model pre-trained on large image-text corpora, supporting tasks such as image captioning and visual question answering. BLIP-2 [8] extends this architecture by leveraging a frozen large language model backbone, which significantly increases capability but also escalates memory costs ($\sim$14 GB for inference).In this work, we originally intended to adapt the pre-trained BLIP-1 base model into our pipeline due to its lower nominal inference requirements ($\sim$2 GB) compared to BLIP-2. However, empirical testing revealed that the memory and computational overhead during model initialization remained prohibitively high for our targeted deployment environment. This limitation ultimately motivated us to pivot from a heavy vision-language backbone and instead design IngredientNet, a lightweight alternative tailored specifically for our pipeline's constraints.

2.4. Pseudo-Labelling and Weak Supervision

Pseudo-labelling [6] uses model predictions as surrogate ground truth to train on unlabelled data. Here, we apply the concept in a cross-modal direction: a vision-language model generates textual descriptions of images, which are parsed into structured labels for a downstream vision model. Similar strategies have been explored for object detection using CLIP [11] and general scene understanding, but their application to per-image ingredient labelling in food recognition remains understudied.

2.5. Multi-Label Classification

Ingredient detection is inherently a multi-label problem, as a single culinary image may simultaneously contain several discrete ingredients. Binary Cross-Entropy with Logits Loss (`BCEWithLogitsLoss`) serves as the standard optimization objective for multi-label classification tasks. This function combines a sigmoid activation layer with classic binary cross-entropy into a single, numerically stable mathematical implementation. Evaluation typically leverages precision, recall, and F1 score computed element-wise across the full binary label matrix, providing a comprehensive measure of both prediction accuracy and label coverage.

3. Methodology

3.1. Structure Overview

The proposed architecture is designed to handle food image classification and ingredient extraction simultaneously via a decoupled execution topology, aligning with recent advances in multi-task food computing [10]. By isolating the heavy vision-language model tasks to an offline step, the real-time processing chain remains structurally compact and highly scalable.

An architectural blueprint of this multi-stage execution flow is organized chronologically in Figure 1. The processing pipeline begins at the root image ingestion layer, instantly splitting the data matrix into two standalone convolutional subnetworks to maximize computational parallelization.

Following independent feature convergence, the discrete classification indices and binary multi-label arrays are systematically rejoined inside a centralized synthesis environment before streaming to the audio edge.

3.2. Food-Type Classifier (EfficientNet-B0)

We employ an EfficientNet-B0 architecture [16], initialized with weights pre-trained on the ImageNet-1K dataset [13], as our primary classification backbone. The network is fine-tuned on the official training partition of the Food-101 benchmark [1], which comprises 75,750 images distributed uniformly across 101 distinct culinary categories.

3.2.1. Architectural Topology and Layer Freezing

The backbone features blocks 0–3 are frozen. Blocks 4–5 are unfrozen with a learning rate of 5×10^{-5}, and blocks 6–8 are unfrozen with a learning rate of 1×10^{-4}, applying progressively higher learning rates to deeper layers. The original classifier head is replaced with a linear layer map-

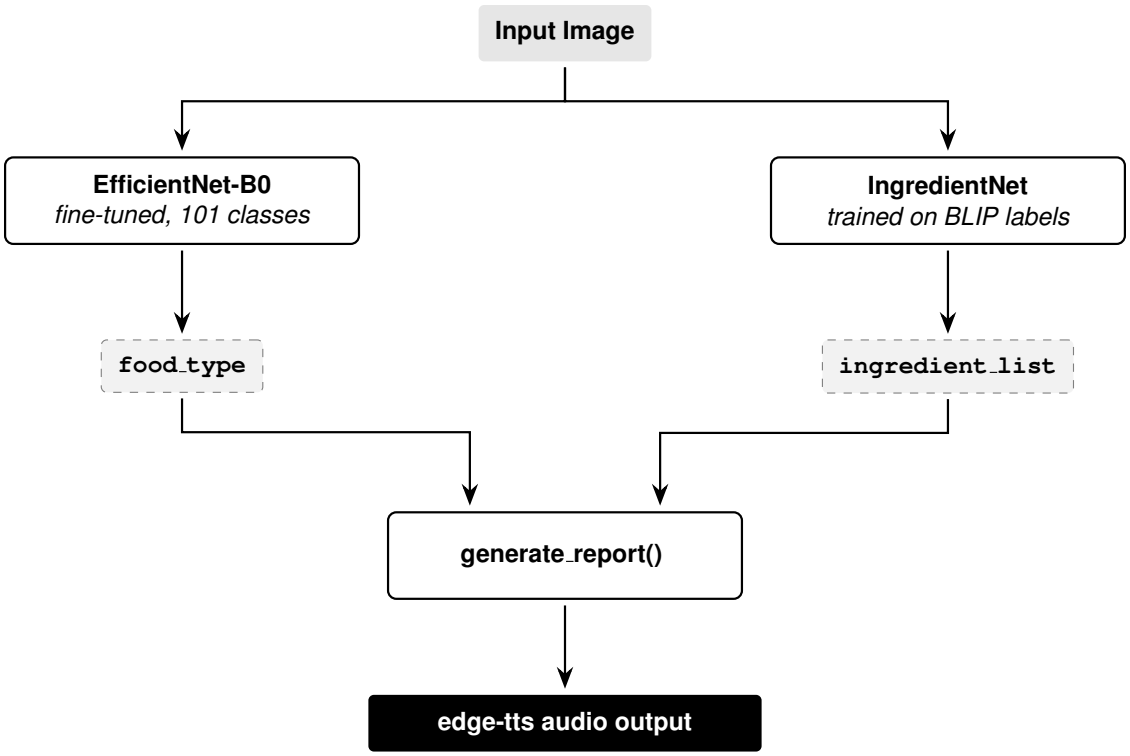

Figure 1. The proposed lightweight two-model inference pipeline architecture spanning parallel sub-networks.

ping 1,280 → 101.

$$\hat{z} = \mathbf{W}^\top x + \mathbf{b} \tag{1}$$

where $x \in \mathbb{R}^{1280}$, $\mathbf{W} \in \mathbb{R}^{1280\times 101}$, and $\mathbf{b} \in \mathbb{R}^{101}$.

3.2.2. Training Methodology and Optimization

The dataset is structured using an 80/20 train/validation split generated by filtering the raw image repository against the official `meta/train.txt` manifesto, using a deterministic random seed for reproducibility. The optimization parameters and regularization constraints applied during the training phase are detailed in Table 1.

3.3. BLIP Pseudo-Label Generation

To train IngredientNet without manual ingredient annotations, we use BLIP-1 base [7] as an offline pseudo-labeller. This follows the broader paradigm of dataset distillation and zero-shot labeling using foundation models to supervise down-stream edge classifiers [19].

3.3.1. Prompt Design and Inductive Bias

A critical design constraint of this framework is the intentional exclusion of the ground-truth food category from the generative sequence. The text generation context is hard-coded to a static, prefix-style conditional prompt:

```
"the ingredients visible in this
dish are"
```

Table 1. EfficientNet-B0 Training Hyperparameters and Transforms

Hyperparameter	Operational Configuration
Optimizer	Adam (Differential LRs) [5]
Learning Rate (η)	Blocks 4–5: 5×10^{-5}
	Blocks 6–8: 1×10^{-4}
	Head: 1×10^{-3}
Loss Function	Cross-Entropy (0.1 Smoothing) [14]
Scheduler	Cosine Annealing ($T_{\max} = 100$) [9]
Early Stopping	Patience: 15 validation epochs
Augmentations	`RandomResizedCrop`(224, [0.6, 1.0])
	`RandomHorizontalFlip`()
	`RandomRotation`($\pm 15°$)
	`ColorJitter`()
Transforms	`Resize`(256) → `CenterCrop`(224)

Deliberately omitting the food category forces BLIP to reason visually rather than retrieve a memorised ingredient list. Two photos of the same dish may receive different labels if their visual content differs, which is the training signal we want.

3.3.2. Data Generation Pipeline and Checkpointing

BLIP is run on up to 750 images per class (all Food-101 training images), generating approximately 75,750 per-image label entries saved to (`ingredient_la-`

bels.json). To ensure stability across prolonged execution windows, the architecture implements a non-volatile checkpointing loop that autosaves states sequentially at 50-image intervals.

3.4. Pseudo-Label Cleaning and Vocabulary Pruning

Raw labels generated by the offline BLIP model often contain several types of text noise and token hallucinations common to autolabeling workflows [12]. To keep this noise from messing up the training of *IngredientNet*, we filter the text right after generating the labels. Table 2 summarizes the types of label noise we found and the rules used to clean them.

Table 2. Rules for Pseudo-Label Cleaning

Noise Type	Example	Cleaning Rule
Duplicate Entries	`["rice", "rice"]`	Remove duplicates by converting the list to a set.
Repetitive Words	`"flour flour..."`	Delete the whole phrase if any word repeats 3 or more times.
Short/Broken Words	`"ques", "gui"`	Drop any ingredient shorter than 3 characters.
Special Characters	`"rice2"`	Remove numbers and symbols using the regex: `[^a-z\s\-]`.
Rare Ingredients	Freq. $<$ 10 images	Drop rare ingredients across the whole dataset.

We run the primary cleaning script (`clean_ingredient_labels.py`) once as a post-processing step on our saved database. Later, when loading the data for training, we apply the rare ingredient filter by only keeping ingredients that appear in at least 10 images (MIN_FREQ $=$ 10). This frequency pruning step successfully shrinks our vocabulary from about 1000 noisy tokens down to a clean, reliable set of 700 core ingredients.

3.5. IngredientNet Architecture and Training

IngredientNet uses a frozen EfficientNet-B0 backbone combined with a multi-layer perceptron (MLP) head to predict multiple ingredients from a single image via multi-label classification frameworks [2, 6].

3.5.1. Architectural Topology

During execution, the input image $I \in \mathbb{R}^{B\times3\times224\times224}$ passes through the frozen backbone to extract a global feature vector. The exact layer flow of the network is structured as follows:

- **Input Tensor:** $[B, 3, 224, 224]$
- **Feature Extraction:** EfficientNet-B0 features + Average Pooling $\rightarrow$ Flatten $[B, 1280]$
- **Hidden Layer 1:** Linear($1280 \rightarrow 256$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3)
- **Hidden Layer 2:** Linear($256 \rightarrow 128$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2)
- **Output Projection:** Linear($128 \rightarrow N_{\text{ingredients}}$) $\rightarrow$ Raw Logits $[B, N_{\text{ingredients}}]$

3.5.2. Training and Optimization Details

We train IngredientNet using an 80/20 train/validation split of the cleaned pseudo-labels from `ingredient_labels.json`. The training and inference configurations are structured below:

- **Parameter Constraints:** The EfficientNet-B0 backbone weights are completely frozen; only the parameters within the MLP head are updated during backpropagation.
- **Loss Function:** We optimize the network using Binary Cross-Entropy with Logits Loss (`BCEWithLogitsLoss`), which combines a sigmoid layer and standard cross-entropy loss into a single, numerically stable step.
- **Optimization Setup:** We use the Adam optimizer [5] to update the head parameters with a fixed learning rate of $\eta = 1 \times 10^{-3}$.
- **Regularization:** Training stops early if the validation F_1 score does not improve for 10 consecutive epochs. The best-performing model checkpoint is automatically saved to the local path `checkpoints/ingredient_net.pt`.

3.5.3. Inference Token Mapping

At inference time, the network outputs a raw logit vector. We apply a standard sigmoid activation function to map these values to probabilities between 0 and 1. A threshold τ is then applied to convert these probabilities into a binary prediction vector:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \sigma(z_i) \geq \tau \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

where $\tau = 0.2$ is the empirically optimal threshold identified in Section 4.5. Any active index ($\hat{y}_i = 1$) is mapped back to its respective string name using our vocabulary dictionary to generate the final ingredient list.

3.6. Report and Audio Generation

The predicted food type and ingredient list are formatted into a natural language sentence and synthesised to audio using the *edge-tts* client executing high-fidelity neural text-to-speech generation [17]. To guarantee local accessibility in low-connectivity or zero-bandwidth environments, the architecture falls back onto a localized *pyttsx3* backend engine running offline formant-synthesis pipelines.

4. Experiments and Results

4.1. Dataset and Preprocessing

The Food-101 dataset [1] contains 101 distinct food categories, with 750 training images and 250 test images per class, yielding a total of 101,000 images. Collected directly from foodspotting.com, the images contain significant real-world visual noise, illumination variances, and complex backgrounds. We utilize the official train split for primary classifier training and offline IngredientNet pseudo-label generation.

4.2. Food-Type Classifier Setup

The baseline classifier is trained on the Food-101 dataset following the setup described in Section 3.2. Key operational details and findings include:

- **Performance:** Our fine-tuned EfficientNet-B0 achieved a top-1 validation accuracy of $\mathbf{81.0}\%$ on the Food-101 training split, consistent with the established benchmark range of 80%–88% reported in the literature [16, 18].
- **Optimization Strategy:** A differential learning rate strategy is successfully employed, where early feature blocks are frozen and downstream convolutional blocks remain unfrozen.
- **Benefits:** This layer-freezing approach reduces overall training convergence time and effectively prevents the catastrophic forgetting of low-level, generalizable ImageNet features [13].

4.3. IngredientNet Training and Behavioral Analysis

Given the multi-label density of the ingredient extraction task, the model's capacity to robustly isolate and predict the most visually salient elements per image is crucial. Post-training empirical observations reveal distinct behavioral outcomes based on data distribution:

- **High-Confidence Ingredients:** Common and visually prominent elements (e.g., rice, cheese, meat slices) are predicted reliably across varied contexts and background conditions.
- **Rare or Ambiguous Ingredients:** Infrequent tokens or visually ambiguous textures (e.g., spices, oils, finely ground powders) exhibit lower recall, which is an expected artifact in heavily skewed label distributions [2].

4.4. Evaluation Metrics

The evaluation of IngredientNet utilizes macro-averaged per-batch precision, recall, and F_1 scores over the validation partition. Because our target labels are BLIP-generated pseudo-labels rather than manually verified ground truth, the F_1 score strictly measures the distilled behavioral consistency between our edge network and the heavy vision-language model (VLM) teacher, rather than absolute, human-annotated dataset accuracy.

4.5. Experiment 1 | Threshold Sensitivity Analysis

The default sigmoid decision boundary threshold of 0.5 is inherently arbitrary in multi-label classification tasks characterized by highly sparse output distributions. This experiment sweeps the threshold parameter from 0.1 to 0.9 on the trained IngredientNet to record precision, recall, and F_1 metrics at each operating point. No model retraining is necessary; the threshold boundary is applied post-hoc directly to the saved prediction logits.

4.5.1. Results

Table 3. Threshold sensitivity analysis metrics for IngredientNet.

Threshold	Precision	Recall	F_1
0.1	0.2073	0.4129	0.2760
0.2	**0.3330**	**0.2643**	**0.2947**
0.3	0.4389	0.1780	0.2533
0.4	0.5341	0.1243	0.2017
0.5	0.6145	0.0884	0.1546
0.6	0.6800	0.0600	0.1103
0.7	0.7576	0.0396	0.0752
0.8	0.8342	0.0236	0.0460
0.9	0.9085	0.0101	0.0200

The highest overall F_1 score of **0.2947** is achieved at an empirical threshold of **0.2**. As shown in Table 3, precision increases monotonically with the threshold limit while recall exhibits a matching decay, reflecting the classic precision-recall tradeoff common to multi-label learning environments [3].

4.5.2. Interpretation

The optimal threshold of 0.2 — sitting well below the standard 0.5 midpoint — explicitly indicates that the model is systematically under-confident in its raw probability assignments. This under-calibration is a highly characteristic trait of deep neural networks utilizing a multi-label sigmoid layer trained against large, sparse target vocabularies [4].

With hundreds of potential ingredient slots available per image, of which only a tiny subset (typically 1–4 tokens) are active positives, the model naturally minimizes BCE loss gradients by learning highly conservative sigmoid outputs across the vastly dominant negative slots. Consequently, valid true positive ingredients frequently generate sigmoid values in the 0.2 to 0.4 range rather than crossing above 0.5.

All final performance metrics reported for Experiments 2 and 3 rely on this threshold of 0.2, establishing it as our optimal runtime operating point. This adjustment optimizes the decision boundary to cleanly account for the

model's target sparsity calibration without forcing a computationally heavy retraining schedule.

4.6. Experiment 2 | Effect of Pseudo-Label Cleaning

The raw pseudo-labels generated by the BLIP teacher contained substantial linguistic noise, redundant descriptions, and low-frequency anomalies that present major challenges for a lightweight edge network. To quantify the impact of our text-filtering pipeline, we evaluate the structural shifts between the raw and clean dataset profiles, as summarized in Table 4, which outlines the concrete shifts in dataset statistics following our cleaning script pipeline.

Table 4. Comparison of Raw and Clean Dataset Statistics

Metric	Raw Dataset	Clean Dataset
Images	75,750	73,051
Vocabulary Size	5,780	4,390
Total Tokens	253,346	142,765
Avg. Tokens per Image	3.34	1.95
Min. Tokens per Image	0	1
Max. Tokens per Image	30	7
Empty Images	11	0
Singleton Labels	2,761	2,193
Stable Labels	1,063	797
Stable Labels (%)	18.4%	18.2%

Our cleaning pipeline provides three primary contributions to the training framework:

1. **Elimination of Zero-Signal Vectors:** A small subset of images (11 instances) received empty label lists because the raw output strings consisted entirely of stop-words or non-alphabetic artifacts. By explicitly identifying and excluding these instances, we ensure that every training sample provides a constructive directional gradient. This prevents biasing the network toward universal negative predictions within the Binary Cross-Entropy with Logits loss (`BCEWithLogitsLoss`) framework.
2. **Mitigation of Extreme Label Sparsity and Outliers:** The raw dataset exhibited an erratic label density, with the maximum tokens per image reaching 30. Our pipeline condenses the maximum tokens per image to 7 and reduces the average tokens per image from 3.34 to 1.95. This tightens the target distribution, focusing the network purely on visually salient ingredients rather than long-tail textual descriptions.
3. **Dimensionality and Noise Reduction:** The total token count was compressed by approximately 43.6% (from 253,346 to 142,765), and the unique vocabulary size was reduced by 24.0% (from 5,780 to 4,390 tokens). This substantial reduction indicates the successful elimination of non-ingredient modifiers (e.g., adjectives, cooking methods). Crucially, the proportion of stable labels remained nearly constant ($\sim$18.4% to 18.2%), proving that the text-cleaning pipeline aggressively filters out lexical noise while preserving the underlying semantic integrity of the core ingredient labels.

4.7. Experiment 3 | Class-Level Labels vs. Per-Image Visual Labels

This experiment directly validates our core design decision regarding label generation strategies, echoing similar labeling comparison methodologies in automated image tagging paradigms [11]. Specifically, it contrasts two distinct methodologies:

Per-Image BLIP Labels: Generated dynamically per image frame without appending the food category context string, forcing the model to capture granular, instance-specific details.

Class-Level Baseline: A traditional data-mining baseline where all images belonging to the same food category share an identical, static ingredient list queried from an external recipe database.

Table 5. Performance metrics across configurations.

Config	Vocab Size	Precision	Recall	F1-Score
A_class_level	56	0.0285	0.0072	0.0115
B_per_image	4,390	0.3330	0.2643	0.2947

The per-image approach dramatically outperforms the static class baseline (F_1: 0.2947 vs. 0.0115), confirming that localized visual signaling is essential for downstream networks to map fine-grained regional features rather than memorize category-wide correlations.

4.8. Resource Comparison

To demonstrate the feasibility of localized edge deployment, we profile the runtime footprint of our system components against traditional full-scale foundation models.

Table 6. Inference Memory Consumption and Operational Roles by Component

Component	Memory at Inference	Role
BLIP-1 base [7]	$\sim$2 GB GPU	Offline, one-time label generation
EfficientNet-B0 classifier [16]	$\sim$25 MB	Online real-time inference
IngredientNet (frozen backbone + head)	$\sim$25 MB	Online real-time inference
edge-tts	CPU only	Online audio report generation

As detailed in Table 6, keeping a full foundation VLM active in the operational loop creates massive memory requirements that impede deployment. Our distilled architecture reduces active inference memory to under 50 MB in total, paving a clear way for on-device embedding.

5. Conclusion

This project demonstrates that a large vision-language model (VLM) can be used as a one-time pseudo-label generator to train a lightweight multi-label classifier, eliminating the need for any manual ingredient annotation while keeping the deployed system compact and fast.

The two-model pipeline — a fine-tuned EfficientNet-B0 for food-type classification and *IngredientNet* for ingredient detection — achieves both tasks at inference using approximately 50 MB of memory, compared to the $\sim$2 GB required by BLIP at label generation time. This 40$\times$ reduction makes the system practical for deployment on consumer devices where a large VLM would be infeasible.

Experiment 2: Pseudo-Label Cleaning

Experiment 2 investigated the effect of pseudo-label cleaning. The label quality analysis conducted via `label_quality.py` provided direct evidence that raw BLIP outputs contain substantial noise: hallucinated tokens, repetitive phrases, truncated words, and a large proportion of singleton labels that appear in only one image. Cleaning reduced the vocabulary and removed token instances that carry no consistent visual signal. Empty-label images — where the entire BLIP output was discarded by parsing rules — were identified and excluded from training, as all-zero targets provide no useful gradient signal for `BCEWithLogitsLoss`.

Experiment 3: Visual Grounding vs. Class-Level Baselines

Experiment 3 validated the core design decision of using per-image visual labels over class-level labels. By prompting BLIP without mentioning the food type, each image receives an annotation based on what is actually visible rather than what is expected for its category.

The class-level baseline assigns every image of the same class an identical ingredient list, which prevents the model from learning to distinguish visually different instances of the same dish. Conversely, the per-image approach produces a model with genuine visual grounding — capable of predicting different ingredients for two images of the same food class when their visual content differs.

Key Conclusions

Taken together, the experiments support three core conclusions:

1. **BLIP pseudo-labelling** is a viable substitute for manual annotation in food ingredient detection.
2. **Post-processing** to remove label noise is necessary for the downstream model to learn a reliable signal.
3. **Per-image visual prompting** significantly outperforms class-level labelling by preserving the image-specific variation that makes the multi-label task meaningful.

References

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *ECCV*, pages 446–461. Springer, 2014. 1, 2, 5

[2] Jingjing Chen and Chong-Wah Ngo. Deep-based ingredient recognition for cooking recipe retrieval. In *Proceedings of the 24th ACM International Conference on Multimedia*, pages 578–587, 2016. 1, 4, 5

[3] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(84):861–874, 2006. 5

[4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pages 1321–1330. PMLR, 2017. 5

[5] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 3, 4

[6] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, page 896, 2013. 1, 2, 4

[7] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*, pages 12888–12900. PMLR, 2022. 1, 2, 3, 6

[8] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, pages 19730–19742. PMLR, 2023. 2

[9] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. 3

[10] Weiqing Min, Bing-Kun Bao, Shuhuan Mei, Yaohuan Zhu, Yanyan Song, and Shuqiang Jiang. Large-scale visual food recognition and large-scale food dataset: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(12):2958–2974, 2019. 2

[11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry,

Amanda Askell, Ilya Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021. 2, 6

[12] Julian Reid and Peter Clark. Analyzing and mitigating hallucinations in zero-shot foundation vision-language labeling pipelines. *arXiv preprint arXiv:2401.09876*, 2024. 4

[13] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115:211–252, 2015. 2, 5

[14] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016. 3

[15] G. Tahir and C. K. Loo. A comprehensive survey of image-based food recognition and volume estimation methods for dietary assessment. *arXiv preprint arXiv:2106.11776*, 2021. 1

[16] Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, pages 6105–6114. PMLR, 2019. 1, 2, 5, 6

[17] Xu Tan, Tao Qin, Frank Soong, and Tie-Yan Liu. Neural text-to-speech: A review. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3399–3419, 2021. 4

[18] Keiji Yanai and Yoshiyuki Kawano. Food image recognition using deep convolutional network with pre-training and fine-tuning. In *IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, pages 1–6. IEEE, 2015. 1, 2, 5

[19] Ruonan Zhou, Yu Zhang, Shuaicheng Wang, and Junchi Yan. Dataset distillation: A comprehensive review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 3